Triplets in haskel VD

Can Valid

March 2026

1 Introduction

1.1 Empty Closure and the "$x = x$" Collapse

A triplet can be *formally closed* without being *semantically productive*. This happens when the triplet is too underspecified to determine anything beyond the survival of its own symbols. In such cases, repeated substitution does not ground the concept; it merely returns the symbol to itself.

A simple Newton-II-style example makes the point clear. Consider the triplet

$$F = ma, \qquad a = \frac{F}{m}, \qquad m = \frac{F}{a}.$$

Suppose only a is independently fixed. Then one may try to recover m from the triplet:

$$m = \frac{F}{a} = \frac{ma}{a} = m.$$

This chain is mathematically consistent, but conceptually revealing: the symbol m has not been determined by the system. It has simply passed through the equations and returned unchanged. The triplet therefore preserves the token without yet grounding its meaning. In this sense, the structure collapses into the form

$$x \rightsquigarrow equationsinvolving x \rightsquigarrow x,$$

which is the schematic content of statements like "mass is just mass" or "that's just what it is."

Haskell-style rendering. A useful analogy is to write the triplet as a mutually recursive Haskell block:

```
a :: Double
a = 2.0

f :: Double
f = m * a

m :: Double
m = f / a
```

This code is syntactically legal Haskell. But if one asks for `m`, evaluation does not produce a new value. The unfolding goes:

```
m
=> f / a
=> (m * a) / a
=> ((f / a) * a) / a
=> (((m * a) / a) * a) / a
=> ...
```

So the program does not terminate with a fresh numerical answer. It keeps unfolding the same dependence structure. At the algebraic level, the same dependence appears more compactly as

```
m
= f / a
= (m * a) / a
= m
```

The two traces show the same phenomenon from different angles:

- the *runtime trace* shows recursive non-termination;
- the *algebraic trace* shows symbolic self-return.

In either case, the point is the same: the triplet has not supplied enough independent structure to fix the concept.

What this shows. This matters because it distinguishes two very different kinds of closure.

Productive closure occurs when the triplet is embedded in a wider scaffold: measurements, state descriptions, equations of motion, detector records, symmetries, or other independently grounded concepts. In that case the triplet participates in a real theory.

Empty closure occurs when the triplet merely allows each symbol to hand itself back through the other two. Then the structure is formally stable, but conceptually thin. It protects the term from contradiction without explaining what the term is doing.

That is why a response like

> "What is energy?"
> "Energy is just energy."

can be philosophically revealing rather than merely lazy. It may indicate that the speaker is relying on a concept held in place by a closed inferential circuit whose effect is structurally similar to the above collapse:

$$x \to roleequations \to x.$$

The concept survives the circle, but its meaning is not deepened by it.

Interpretive proposal. Whenever scientific discourse falls back on remarks like

- "that's just what force is,"
- "energy is just the conserved quantity,"
- "mass is just what appears there,"

a process of this kind may be occurring, either implicitly or as a logical consequence of the underlying inferential structure. The concept is not necessarily meaningless, but the local explanatory move has become one of *token-preservation rather than grounding*. The speaker is not adding new structure; they are returning the symbol to itself through an accepted loop.

The lesson is therefore:

> Formal closure is not yet semantic grounding.

A triplet may be algebraically closed and still fail to answer the question it appears to answer. The mere fact that three terms interdefine one another does not by itself guarantee that the resulting closure is scientifically meaningful rather than empty.